QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 Team Boost – KitaVisa*
Team Repo URL	https://github.com/ciwahua/kita-visa
Live Deployment URL (optional for preliminary round)	-

Objective:

Ensure that KitaVisa can reliably classify visa intent, generate accurate step-by-step workflows, detect missing information, and handle user inputs (including documents and ambiguous queries) while maintaining system stability and acceptable response time. The system must also ensure that GLM outputs are structured, validated, and aligned with the knowledge base.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- Visa Intent Classification: AI extracts visa type (Student, Work, Dependent, Social Visit) from user input
- AI Chat Assistant: Conversational interface that guides users step-by-step
- Gap Analysis: Identifies missing visa requirements after confirmation
- Document Upload & Validation: Parses and validates uploaded files (PDF, DOCX, TXT)
- Session-Based Context: Stores user state (visa type, gaps, intent) across requests

1.2 Out-of-Scope

- Integration with immigration systems
- Real-time visa status tracking
- Payment systems
- Native mobile app

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

*The risk are to be evaluated using the 5*5 Risk assessment Matrix. Risk Score = Likelihood * Severity.*

Technical Risk	Likelihood	Severity	Score	Mitigation	Testing Approach
AI gives wrong visa classification	2	4	8 (medium)	Add confirmation step before proceeding	Test ambiguous inputs
AI returns unstructured / invalid response	4	4	16 (Critical)	Enforce structured prompts + fallback handling	Validate JSON before returning
Stateless AI (loses context)	4	5	20 (Critical)	Implement sessionStore for context persistence	Multi-turn conversation testing
File upload fails (e.g. multer missing)	3	4	12 (High)	Dependency validation + install checks	Run upload tests
GLM API failure / timeout	3	5	15 (High)	Retry logic + fallback response	Simulate API downtime
Missing package errors (e.g. multer)	4	3	12 (High)	npm install + dependency check	Local environment testing
Incorrect gap analysis	3	4	12 (High)	Validate output format and logic	Compare expected vs actual gaps
Slow response time (>20s)	3	3	9 (Medium)	Optimize prompt size	Load testing

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - *Scope:*
    - i. *extractIntent()*
    - ii. *analyzeGaps()*
    - iii. *validateDocument()*
    - iv. *sessionStore (set/get)*
  - *Execution:*
    - i. *Run locally during development*
    - ii. *Triggered on backend updates*
  - *Isolation:*
    - i. *GLM API mocked where needed*
    - ii. *Pass Condition:*
    - iii. *Correct visa type returned*
    - iv. *Valid JSON structure*
    - v. *No crashes*
- **Integration Test**
  - *Scope:*
    - i. */api/query*
    - ii. */api/analyze-gaps*
    - iii. */api/chat*
    - iv. *File upload flow*
  - *Execution:*
    - i. *After merging frontend + backend*
    - ii. *Real API calls (GLM used)*
  - *Pass Condition:*
    - i. *End-to-end flow works*
    - ii. *Context persists*
    - iii. *No broken response*
- **Test Environment (CI/CD Practice):**
  - *Local: localhost:3001 backend + React frontend*
  - *API: GLM external API*
  - *CI/CD GitHub push testing (manual for MVP)*

- **Regression Testing & Pass/Fail Rules:**
  - *Run after every major change:*
    - i. *chat logic*
    - ii. *session handling*
    - iii. *AI prompt updates*
  - ***Pass Rule**:*
    - i. *All previous working flows still work*

© UMHackathon 2026

Email: umhackathon@um.edu.my

        ii. *No breaking changes*
    - *need to meet the threshold to allow continue for further testing]*

- **Test Data Strategy:**
    - *Manual inputs:*
        i. *"I want to study in Malaysia"*
        ii. *"I got a job offer"*
        iii. *"I want to visit my family"*
    - *Documents:*
        i. *Sample PDF offer letter*
        ii. *Fake passport text*
        iii. *Empty file (edge case)*

- **Passing Rate Threshold:**
    - *[e.g., A minimum of 86.2% of all test cases that are executed must pass. For the critical tests, all the tests (100%) need to be passed and there is no exception for that. Because critical failure costs huge in production]*

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Check	***Requirement***	***Result***	***Status***
Build	*No errors*	*0 errors*	*Passed*
Unit Tests	*100%*	*92%*	*Partial*
API Response	*Valid JSON*	*Mostly valid*	*Partial*
Linting	*Clean*	*Minor issues*	*Partial*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Check	***Requirement***	***Result***	***Status***
Regression	*≥90%*	*88%*	*Partial*
AI Output Accuracy	*≥80%*	*85%*	*Passed*
Critical Bugs	*0*	*1 (context bug)*	*Failed*
API Performance	*< 800ms*	*~600ms*	*Pass*

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-01	Happy Case (Entire Flow): Visa Workflow Generation	Verify that a user can go through the full visa process from query, classification, confirmation, gap analysis, workflow display	1. Open web app 2. Enter: "I got accepted into Universiti Malaya" 3. System suggests Student Pass 4. User confirms 5. System triggers gap analysis 6. Dashboard displays missing documents	System correctly identifies visa type, confirms with user, generates structured gap list, and displays checklist workflow	System correctly classified Student Pass, confirmation step triggered, and missing documents displayed in dashboard	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

TC-02	Negative Case: Ambiguous Input Handling	Verify system handles unclear or vague user input without crashing or misguiding	1. Enter: “I want to go Malaysia” 2. Submit query	System should ask for clarification instead of guessing visa type	System prompts user to clarify purpose (study/work/visit), no incorrect classification made	**Passed**
TC-03	Edge Case: Missing Session Context	Verify system behavior when sessionId/context is missing or not stored	1. Send message without sessionId 2. Ask: “What visa do I need?”	System should fallback safely and ask for required details instead of assuming context	System responds with clarification request, does not crash or hallucinate context	**Passed**
TC-04	Negative Case: File Upload Error	Verify system handles unreadable or invalid file uploads	1. Upload corrupted/empty PDF 2. Submit for validation	System should reject file and show error message without crashing	System returns “Could not read file content” and continues running normally	**Passed**
TC-05	NFR (Performance): API Response Time	Verify backend API responds within acceptable time under normal load	1. Send 20 requests to /api/query 2. Measure response time	Average response time < 3 seconds, no errors	Average response time ~1.8s, no failures observed	**Passed**
TC-06	NFR (Reliability): GLM Failure Handling	Verify system handles AI API failure gracefully	1. Simulate GLM API failure (invalid API key / timeout) 2. Send chat request	System should return fallback message and not crash	System returns “Sorry, I had trouble responding. Please try again.”	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

6.1 Prompt Tests

ID	Input	Expected	Actual	Status
AI-01	“I want to study in Malaysia”	Student Pass	Student Pass	Pass
AI-02	“I want to work in KL”	Employment Pass	Employment Pass	Pass
AI-03	“Tell me about my visa”	Uses context	Asked again	Fail

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Field	Details
Max Input	~5000 characters
Test Input	10,000+ characters
Expected	Truncate / handle
Actual	Partial failure
Status	Partial

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

Test ID	Prompt Input	Expected Output	Actual Output	Status
AI-04	"I want something idk maybe study or work or visit idk"	System should detect ambiguity in user intent and request clarification instead of making a visa classification decision	System correctly identifies low-confidence input and triggers clarification prompt: "Could you clarify your purpose in Malaysia? (Study, Work, Family, Visit, or Dependent)"	**Passed**

6.4. Hallucinating Handling

This section is defined for you to describe how your system does detect or mitigate hallucinating response.

a. Strict System Prompt Constraints

The GLM is constrained using a highly structured system prompt that defines:

- Valid visa categories (Student Pass, Employment Pass, Dependent Pass, Social Visit Pass)
- Strict instruction to NOT guess visa types without sufficient confidence
- Requirement to rely only on provided user context and system-confirmed data
- Prohibition of free-form decision making without confirmation step

This reduces the probability of the model generating unsupported visa recommendations.

b. Controlled Output Design (Schema-Grounded Responses)

All AI outputs are constrained to expected formats:

- Intent classification → single visa type only
- Gap analysis → structured list of missing documents
- Chat response → plain text only (no JSON or structured fields in chat mode)

© UMHackathon 2026

Email: umhackathon@um.edu.my

This ensures the model cannot introduce arbitrary or unstructured information that breaks system logic.

c. Backend Validation Layer

Before any AI response is returned to the frontend:

- Responses are checked for missing or invalid fields
- Empty or malformed outputs are rejected and replaced with fallback responses
- Visa types are validated against a predefined mapping table

If inconsistency is detected, the system overrides the AI output with safe defaults (e.g., "Unknown visa type" or request for clarification).

d. Session-Based Context Control

The system uses session storage to maintain:

- Confirmed visa type
- Extracted intent
- Gap analysis results

However, context is only injected after confirmation. This prevents the model from making premature assumptions based on incomplete user input.

e. Confirmation-Based Decision Gate

A critical anti-hallucination mechanism is the two-step flow:

- AI suggests visa type based on intent classification
- User must explicitly confirm before any gap analysis is executed

This ensures the model cannot directly proceed to final conclusions without human validation.

f. **Fallback & Safe Response Strategy**

When uncertainty or failure is detected:

- The system returns a neutral response such as:
  "Could you clarify your purpose in Malaysia (Study, Work, Visit, or Dependent)?" to prevent the model from fabricating assumptions when confidence is low.

Important Note:

© UMHackathon 2026

Email: umhackathon@um.edu.my

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my